

Smart Teacher — File-by-File Deep Dive

Documentation détaillée fichier-par-fichier, ligne-par-ligne (ou fonction-par-fonction) des modules les plus critiques du projet. Couvre les calculs de scores, la logique de personnalisation, le pipeline LLM, le cache, le pause/resume, et le FSM. Note : 166 fichiers Python total dans le projet (~50 000 lignes). Ce document couvre les **~25 fichiers les plus importants** (les autres sont du glue code, tests, ou helpers triviaux). Pour un fichier non couvert, voir [SMART_TEACHER_ARCHITECTURE.md](#).

Sommaire

Configuration & boot 1. [core/config.py](#) 2. [main.py](#)

LLM stack 3. [ai/llm_router.py](#) 4. [ai/llm.py](#) (Brain) 5. [ai/local_llm.py](#)

Auth & sécurité 6. [handlers/auth.py](#) 7. [routes/auth.py](#)

FSM dialogue & session 8. [pedagogy/dialogue.py](#)

Pédagogie — scores 9. [pedagogy/mastery_repo.py](#) 10. [pedagogy/review_scheduler.py](#) (FSRS) 11. [pedagogy/engagement.py](#) 12. [pedagogy/student_knowledge.py](#)

Confusion 13. [pedagogy/confusion/fusion.py](#) 14. [pedagogy/confusion/detector.py](#)

Personnalisation 15. [pedagogy/personalization/profile.py](#) 16. [pedagogy/personalization/tts_adapter.py](#) 17. [pedagogy/personalization/bandit/thompson.py](#) 18. [pedagogy/personalization/bandit/reward.py](#)

Resume / pause 19. [pedagogy/resume_intelligence.py](#) 20. [services/presentation.py](#)

RAG 21. [rag/multimodal_rag.py](#) (excerpts)

Knowledge Graph 22. [pedagogy/knowledge_graph/graph.py](#) 23. [pedagogy/concept_from_titles.py](#)

Agentic graphs 24. [agentic/qa/intent.py](#) 25. [agentic/qa/responder.py](#) (excerpts) 26. [agentic/qa/reviewer.py](#) 27. [agentic/teaching/narrator.py](#)

WebSocket 28. [handlers/ws.py](#) (excerpts)

1. core/config.py

Rôle : configuration centralisée. Toutes les variables d'environnement sont lues ici, avec valeurs par défaut sensées. Importée partout (`from core.config import Config`).

1.1 Pattern général

```
class Config:
```

```
VARIABLE_NAME: TypeAnnotation = os.getenv("VARIABLE_NAME", "default_value")
```

Tous les attributs sont **class-level** (pas d'instance) — accès direct **Config.X**.

1.2 API Keys & LLM (lignes 24-35)

```
OPENAI_API_KEY: Optional[str] = os.getenv("OPENAI_API_KEY")
ELEVENLABS_API_KEY: Optional[str] = os.getenv("ELEVENLABS_API_KEY")
GROQ_API_KEY: Optional[str] = os.getenv("GROQ_API_KEY")
GROQ_BASE_URL: str = os.getenv("GROQ_BASE_URL", "https://api.groq.com/openai/v1")
GROQ_MODEL: str = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")
```

- Si **OPENAI_API_KEY** est **None** → **LLMRouter** skip OpenAI au boot.
- **GROQ_BASE_URL** est l'endpoint OpenAI-compatible de Groq (passe **langchain_openai.ChatOpenAI(base_url=...)**).
- **GROQ_MODEL** par défaut = le plus puissant gratuit. Alternatives : **llama-3.1-8b-instant** (rapide), **gemma2-9b-it**.

1.3 Database (lignes 37-47)

```
POSTGRES_HOST: str = "localhost"
POSTGRES_PORT: int = 5432
POSTGRES_DB: str = "smart_teacher"
POSTGRES_USER: str = "admin"
POSTGRES_PASSWORD: str = "secret"
DATABASE_URL: str = f"postgresql+asyncpg://{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"
```

URL construit dynamiquement → utilise asyncpg driver pour SQLAlchemy async.

1.4 TTLs (lignes 54-80)

Sémantique de chaque TTL :

- **SESSION_TOKEN_TTL=300s (5min)** : token WS one-time, court (juste le temps de la connexion initiale)
- **HTTP_HISTORY_TTL=3600s (1h)** : historique conversationnel REST **/ask**
- **SESSION_TTL=3600s (1h)** : état dialogue WS — « rallumer son onglet en revenant du déjeuner reprend où on en était »
- **PRESENTATION_SNAPSHOT_TTL=3600s (1h)** : alias de **SESSION_TTL**
- **TTS_CACHE_TTL=86400s (24h)** : audio TTS pré-généré, peut survivre à la session
- **NARRATION_CACHE_TTL=604800s (7j)** : narration LLM cross-session par slide
- **EMBEDDING_CACHE_TTL=86400s (24h)** : embeddings BGE/OpenAI

■ ■ **Drift potentiel** : **NARRATION (7j)** > **SESSION (1h)** → une narration peut être servie à un nouvel étudiant après que la session originale est morte. C'est intentionnel (cache cross-session par design).

1.5 RAG (lignes 104-136)

Champs critiques :

```
RAG_NUM_RESULTS = 5          # top-K final après rerank
```

```
RAG_USE_IDEA_CHUNKING = false    # ■■ DEFAULT OFF – Ollama hallucine
RAG_USE_GRAPH_EXPANSION = true   # KG-augmented retrieval
KG_DISABLE_LLM_ENRICH = true     # ■■ DEFAULT TRUE – anti-hallucination
```

Pourquoi `RAG_USE_IDEA_CHUNKING=false` par défaut : Ollama (le fallback quand OpenAI off) hallucine régulièrement sur ce prompt — il invente des définitions ("RI = Research Interface") qui finissent dans Qdrant et corrompent le RAG. À ré-activer uniquement avec un LLM hosté fiable (OpenAI, Anthropic, Groq).

1.6 Engagement weights (lignes 298-316)

```
ENGAGEMENT_W_RECENTCY = 0.30
ENGAGEMENT_W_QUESTION = 0.25
ENGAGEMENT_W_CONFUSION = 0.20
ENGAGEMENT_W_PASSIVE = 0.15
ENGAGEMENT_W_INTERRUPT = 0.10
# Sum = 1.00 (assert vérifié au boot par engagement.py)
```

1.7 validate() classmethod (lignes 330-353)

```
@classmethod
def validate(cls) -> None:
    errors = []
    if not cls.OPENAI_API_KEY and not cls.DISABLE_OPENAI:
        errors.append("OPENAI_API_KEY missing in .env (set DISABLE_OPENAI=true to use Ollama only)")
    if cls.TTS_PROVIDER == "elevenlabs" and not cls.ELEVENLABS_API_KEY:
        errors.append("ELEVENLABS_API_KEY missing")
    if errors:
        # print + sys.exit(1)
```

Appelée au boot dans `main.py`. Si `errors` non vide → process s'arrête avec exit 1.

2. main.py

Rôle : entry point FastAPI + uvicorn. Boot sequence, routes mounting, middleware, lifespan.

2.1 Boot sequence (résumée)

```
1. Config.validate()                                # crash si OPENAI manquant et pas DISABLE
2. Setup logging (RotatingFileHandler)
3. app = FastAPI(lifespan=lifespan)
4. app.add_middleware(CORSMiddleware, ...)
5. app.middleware("http")(static_html_auth_gate)    # JWT gate sur /static/*.html
6. app.mount("/static", StaticFiles(...))
7. app.mount("/media", StaticFiles(...))
8. app.include_router(auth.router)                  # /auth/...
9. app.include_router(course.router)                # /course/...
10. app.include_router(session.router)              # /session/...
11. app.include_router(student.router)              # /student/...
12. ... (autres routers)
13. app.add_api_route("/", index_redirect)
```

2.2 lifespan (async context manager)

```
@asynccontextmanager
```

```

async def lifespan(app: FastAPI):
    # STARTUP
    log.info("■ Interface UI : http://localhost:8000/static/index.html")
    await create_tables() # CREATE TABLE IF NOT EXISTS
    await run_lazy_migrations() # ALTER TABLE ADD COLUMN IF NOT EXISTS
    diagnostics.print_service_health() # ping pg/redis/qdrant/ES/CH/MinIO
    Config.print_info()
    deps.init_singletons() # rag, voice, brain, dialogue, transcriber

    yield

    # SHUTDOWN
    await deps.close_singletons()
    log.info("■ Smart Teacher shutdown")

```

2.3 static_html_auth_gate (lignes 296-329)

Middleware qui protège `GET /static/*.html` (sauf `login.html`) :

```

@app.middleware("http")
async def static_html_auth_gate(request, call_next):
    path = request.url.path
    if (request.method == "GET"
        and path.startswith("/static/")
        and path.endswith(".html")
        and path not in _PUBLIC_HTML_PAGES):

        # 1. Recherche token cookie d'abord, puis header
        token = request.cookies.get("smart_teacher_token", "")
        if not token:
            auth_header = request.headers.get("Authorization", "")
            if auth_header.startswith("Bearer "):
                token = auth_header[7:]

        # 2. Pas de token → redirect login
        if not token:
            log.warning("■ GATE REDIRECT | path=... | reason=no_token")
            return RedirectResponse(url="/static/login.html", status_code=302)

        # 3. Token invalide/expiré → redirect login
        try:
            from handlers.auth import decode_access_token
            claims = decode_access_token(token)
            log.info("■ GATE PASS | path=... sub=... role=...")
        except Exception as exc:
            log.warning("■ GATE REDIRECT | reason=invalid_token (%s)", exc)
            return RedirectResponse(url="/static/login.html", status_code=302)

    return await call_next(request)

```

`_PUBLIC_HTML_PAGES = {"/static/login.html"}` — seule page accessible sans auth.

3. ai/llm_router.py

Rôle : router LLM unifié OpenAI ↔ Groq ↔ Ollama avec fallback automatique.

3.1 LLMStats (dataclass, lignes 40-62)

```
@dataclass
```

```

class LLMStats:
    openai_calls: int = 0
    openai_errors: int = 0
    groq_calls: int = 0
    groq_errors: int = 0
    ollama_calls: int = 0
    ollama_errors: int = 0
    fallback_events: int = 0  # # de turns où le préféré a échoué
    both_failed: int = 0  # # de turns où tout a échoué
    _lock: Lock = ...

    def snapshot(self) -> dict:
        with self._lock:
            return {field: value for field, value in self.__dict__.items() if not field.startswith("_")}

```

Threadsafe (utilisé depuis ThreadPoolExecutor pour ingestion).

3.2 LLMRouter.__init__

```

def __init__(self, openai_model="gpt-4o-mini", ollama_model="mistral", ollama_url="http://localhost:11434"):
    self.openai_model = openai_model
    self.ollama_model = ollama_model
    self.ollama_url = ollama_url.rstrip("/")
    self._openai_disabled_reason: Optional[str] = None
    self._disable_lock = Lock()
    self.stats = LLMStats()

    # Groq config – read at init
    self._groq_enabled: bool = False
    self._groq_model: str = "llama-3.3-70b-versatile"
    self._groq_base_url: str = "https://api.groq.com/openai/v1"
    self._groq_api_key: Optional[str] = None

    try:
        from core.config import Config
        if getattr(Config, "DISABLE_OPENAI", False):
            self._openai_disabled_reason = "DISABLE_OPENAI=true (config)"
            log.info("LLMRouter : OpenAI désactivé via Config.DISABLE_OPENAI")
        self._groq_api_key = getattr(Config, "GROQ_API_KEY", None) or None
        self._groq_model = getattr(Config, "GROQ_MODEL", self._groq_model)
        self._groq_base_url = getattr(Config, "GROQ_BASE_URL", self._groq_base_url)
        self._groq_enabled = bool(self._groq_api_key)
        if self._groq_enabled:
            log.info("LLMRouter : ■ Groq activé | model=%s base=%s ...", ...)
        else:
            log.info("LLMRouter : Groq désactivé (GROQ_API_KEY manquant)")
    except Exception:
        pass

```

Au boot, log visible :

```
LLMRouter : ■ Groq activé | model=llama-3.3-70b-versatile base=https://api.groq.com/openai/v1
```

3.3 invoke() — fallback chain

```

def invoke(self, prompt, prefer="openai", temperature=0.0, max_tokens=900):
    prefer = prefer.lower().strip()
    if prefer not in {"openai", "groq", "ollama"}:
        prefer = "openai"

    # Ordre de fallback selon prefer
    if prefer == "openai":
        chain = ["openai", "groq", "ollama"]
    elif prefer == "groq":
        chain = ["groq", "openai", "ollama"]

```

```

else: # ollama
    chain = ["ollama", "groq", "openai"]

first_try = chain[0]
first_failed = False
for idx, backend in enumerate(chain):
    text = self._call(backend, prompt, temperature, max_tokens)
    if text:
        if first_failed:
            log.info(f"LLM fallback {first_try} → {backend} succeeded")
            return text
        if idx == 0:
            first_failed = True
            with self.stats._lock:
                self.stats.fallback_events += 1

# All failed
with self.stats._lock:
    self.stats.both_failed += 1
log.warning(...)
return None

```

Note importante : on ne pré-filtre PAS les backends dans la chain — chaque `_call_*` court-circuite si non disponible. Ça préserve le contrat des tests qui mockent `_call_openai` directement.

3.4 `_call_openai`

```

def _call_openai(self, prompt, temperature, max_tokens) -> Optional[str]:
    if self._openai_disabled_reason:
        return None # short-circuit
    try:
        from langchain_openai import ChatOpenAI
        from langchain_core.messages import HumanMessage
        with self.stats._lock:
            self.stats.openai_calls += 1
        llm = ChatOpenAI(
            model=self.openai_model,
            temperature=temperature,
            max_tokens=max_tokens,
            max_retries=0, # let our router handle retries
        )
        response = llm.invoke([HumanMessage(content=prompt)])
        return (response.content or "").strip() or None
    except Exception as exc:
        with self.stats._lock:
            self.stats.openai_errors += 1
        if self._is_permanent_openai_error(exc):
            self.disable_openai(str(exc)) # quota exhaustion → disable for session
        return None

```

Erreurs permanentes (déclenchent `disable_openai`) :

```

_OPENAI_PERMANENT_ERROR_TOKENS = ("quota", "rate limit", "ratelimit", "429",
                                   "insufficient_quota", "authentication",
                                   "invalid_api_key")

```

3.5 `_call_groq`

```

def _call_groq(self, prompt, temperature, max_tokens) -> Optional[str]:
    if not self._groq_enabled:
        return None
    import time as _t
    try:
        from langchain_openai import ChatOpenAI # OpenAI-compatible API
        from langchain_core.messages import HumanMessage

```

```

with self.stats._lock:
    self.stats.groq_calls += 1
t0 = _t.time()
log.info("■ GROQ generate START | model=%s | prompt_chars=%d (~%d tokens) | temp=%.2f max_tokens=%d",
        self._groq_model, len(prompt), len(prompt) // 4, temperature, max_tokens)
llm = ChatOpenAI(
    model=self._groq_model,
    api_key=self._groq_api_key,
    base_url=self._groq_base_url,          # ← repointe vers Groq
    temperature=temperature,
    max_tokens=max_tokens,
    max_retries=0,
)
response = llm.invoke([HumanMessage(content=prompt)])
elapsed = _t.time() - t0
text = (response.content or "").strip()
log.info("■ GROQ generate DONE | model=%s | took=%.2fs | out_chars=%d",
        self._groq_model, elapsed, len(text))
return text or None
except Exception as exc:
    with self.stats._lock:
        self.stats.groq_errors += 1
    log.warning(f"■ GROQ invoke failed: {exc}")
    return None

```

L'astuce clé : `langchain_openai.ChatOpenAI` accepte un `base_url` custom → on peut pointer vers Groq qui expose une API OpenAI-compatible.

3.6 _call_ollama

```

def _call_ollama(self, prompt, temperature, max_tokens) -> Optional[str]:
    try:
        import requests
        with self.stats._lock:
            self.stats.ollama_calls += 1
        options = {"temperature": temperature, "num_predict": max_tokens}
        try:
            from core.config import Config as _Cfg
            n_threads = int(getattr(_Cfg, "OLLAMA_NUM_THREADS", 0) or 0)
            if n_threads > 0:
                options["num_thread"] = n_threads          # ← honor OLLAMA_NUM_THREADS
        except Exception:
            pass
        payload = {
            "model": self.ollama_model,
            "prompt": prompt,
            "stream": False,
            "options": options,                            # ← model params dans options dict
        }
        response = requests.post(
            f"{self.ollama_url}/api/generate",
            json=payload,
            timeout=None,                                   # ← Ollama CPU peut être lent
        )
        if response.status_code != 200:
            with self.stats._lock:
                self.stats.ollama_errors += 1
            return None
        return (response.json().get("response", "") or "").strip() or None
    except Exception as exc:
        with self.stats._lock:
            self.stats.ollama_errors += 1
        return None

```

Important : `timeout=None` parce qu'Ollama CPU peut prendre 60-600s sur des prompts complexes. Les callers parallèles bornent leur propre patience.

3.7 disable_openai (idempotent)

```
def disable_openai(self, reason: str) -> None:
    with self._disable_lock:
        if not self._openai_disabled_reason:
            self._openai_disabled_reason = reason
            log.warning(f"OpenAI disabled : {reason}")
```

Première raison enregistrée gagne (idempotent). Reset = restart process.

3.8 Singleton get_default_router()

```
_default_router: Optional[LLMRouter] = None
_default_lock = Lock()

def get_default_router() -> LLMRouter:
    global _default_router
    if _default_router is None:
        with _default_lock:
            if _default_router is None:
                _default_router = LLMRouter()
    return _default_router
```

Pattern double-checked locking. Utilisé partout dans le code (`from ai.llm_router import get_default_router`).

4. ai/llm.py (Brain)

Rôle : classe principale pour Q&A direct + présentation de slides. ~1800 lignes (le plus gros fichier après ws.py + multimodal_rag.py).

4.1 Brain.__init__ (lignes 380-403)

Setup OpenAI client OU Groq client selon config :

```
def __init__(self):
    self.client: Optional[OpenAI] = None
    self.history: list = []
    self.max_history_len = Config.MAX_HISTORY_TURNS * 2
    self.session_throttlers: dict[str, dict] = {}
    self.min_call_interval = 1.0 # 1 second minimum between calls
    self.max_calls_per_minute = 10

    self._client_model: str = Config.GPT_MODEL
    self._client_provider: str = "none"

    if Config.DISABLE_OPENAI and getattr(Config, "GROQ_API_KEY", None):
        # Repoint self.client at Groq's OpenAI-compatible endpoint
        try:
            self.client = OpenAI(
                api_key=Config.GROQ_API_KEY,
                base_url=Config.GROQ_BASE_URL,
                max_retries=0,
            )
            self._client_model = Config.GROQ_MODEL
            self._client_provider = "groq"
            log.info("■ Brain : Groq activé (OpenAI désactivé) | model=%s base=%s", ...)
        except:
```

```

        except Exception as exc:
            log.error(f"■ Groq client init failed: {exc} → Ollama only")
    elif Config.DISABLE_OPENAI:
        log.info(f"■ DISABLE_OPENAI=true (et GROQ_API_KEY absent) → Brain runs Ollama-only")
    elif Config.OPENAI_API_KEY:
        try:
            self.client = OpenAI(api_key=Config.OPENAI_API_KEY, max_retries=0)
            self._client_provider = "openai"
            log.info(f"■ OpenAI API clé valide")
        except Exception as exc:
            log.error(f"■ OpenAI erreur: {exc}")
    else:
        log.info(f"■ OpenAI non configuré (fallback Ollama utilisé)")

    self.fallback = LocalLLMFallback(model="mistral") # Ollama

```

L'astuce critique : quand `DISABLE_OPENAI=true` + `GROQ_API_KEY` set, on repointe `self.client` (l'instance OpenAI du SDK) vers Groq. Tous les appels `self.client.chat.completions.create(...)` plus loin dans le fichier marchent transparent grâce à la compatibilité d'API.

4.2 `_check_rate_limit` (lignes 430-477)

Throttler par session_id pour empêcher le LLM-spam :

```

def _check_rate_limit(self, session_id: str | None = None) -> tuple[bool, str]:
    if not session_id:
        return True, ""

    now = time.time()
    if session_id not in self.session_throttlers:
        self.session_throttlers[session_id] = {
            "last_call_time": now,
            "call_count": 0,
            "minute_reset_time": now,
        }
        return True, ""

    throttler = self.session_throttlers[session_id]

    # 1. Min interval entre 2 calls
    time_since_last = now - throttler["last_call_time"]
    if time_since_last < self.min_call_interval:
        return False, f"Rate limited: wait {self.min_call_interval - time_since_last:.1f}s"

    # 2. Max calls par minute
    if now - throttler["minute_reset_time"] > 60:
        throttler["call_count"] = 0
        throttler["minute_reset_time"] = now
    if throttler["call_count"] >= self.max_calls_per_minute:
        return False, f"Per-minute limit reached ({self.max_calls_per_minute} calls/min)"

    # OK – update + autorise
    throttler["last_call_time"] = now
    throttler["call_count"] += 1
    return True, ""

```

Limites : 1 call/sec par session, 10 calls/min. Empêche un client buggué d'écraser le LLM.

4.3 `ask()` — Q&A method (lignes 596+)

Pipeline en 3 niveaux de fallback :

```

def ask(self, question, course_context="", reply_language=None, ...) -> tuple[str, float]:

```

```

allowed, reason = self._check_rate_limit(session_id)
if not allowed:
    return "Trop rapide!", 0.0

start = time.time()
lang = (reply_language or "en").lower()[:2]

# Build system prompt (course-bound, persona, math notation rules, ...)
system_content = get_system_prompt(domain, lang)
if course_context:
    system_content += f"\n\nCOURSE CONTEXT:\n{course_context}"

messages = [
    {"role": "system", "content": system_content},
    *self.history,
    {"role": "user", "content": question},
]

# 1■■ OpenAI / Groq (via self.client)
if self.client:
    try:
        log.info("■■ Tentative OpenAI...") # log dit "OpenAI" même si en fait Groq
        response = self.client.chat.completions.create(
            model=self._client_model, # ← "gpt-4o-mini" OU "llama-3.3-70b-versatile"
            messages=messages,
            temperature=Config.GPT_TEMPERATURE,
            max_tokens=Config.GPT_MAX_TOKENS,
        )
        answer = self._clean_for_speech(response.choices[0].message.content)
        answer = self._dedupe_answer_text(answer)
        self.history.append({"role": "user", "content": question})
        self.history.append({"role": "assistant", "content": answer})
        if len(self.history) > self.max_history_len:
            self.history = self.history[2:]
        duration = time.time() - start
        log.info(f"■■ OpenAI OK | {duration:.2f}s | lang={lang} | {len(answer)} chars")
        return answer, duration
    except Exception as openai_err:
        if self._should_disable_openai(openai_err):
            self._disable_openai(str(openai_err))
            log.warning(f"■■ OpenAI échoué: {openai_err} → Tentative Ollama...")

# 2■■ Ollama fallback (toujours dispo)
if self.fallback and self.fallback.available:
    # ... HTTP call to localhost:11434 ...
    # ... return answer ...

# 3■■ Ultime fallback : message d'erreur
return _FALLBACK_ERROR_MESSAGE[lang], time.time() - start

```

4.4 _clean_for_speech (lignes ~2792)

Post-processeur qui nettoie le texte LLM avant TTS :

```

@staticmethod
def _clean_for_speech(text: str, language: str = "fr") -> str:
    # Strip markdown
    text = re.sub(r"^[^*]+\*", r"\1", text)
    text = re.sub(r"^\^#\s*", "", text, flags=re.M)
    text = re.sub(r"``([^\`]+)`", r"\1", text)
    # Strip LaTeX
    text = re.sub(r"\$+([^\$]+)\$", r"\1", text)
    text = re.sub(r"\\[a-zA-Z]+\{([^\}]+)\}", r"\1", text)
    # Math notation → plain spoken
    if language == "fr":
        text = text.replace("=", " égale ")
        text = text.replace("→", " donne ")
    # ... (continue pour 100+ lignes)

```

```
return text.strip()
```

Voir aussi [audio/math_speech.py](#) qui est un post-processeur dédié plus complet (full unicode + LaTeX).

4.5 present() — slide narration (lignes 953+)

Méthode dédiée à la narration TTS d'une slide. Construit un prompt PRESENTATION (vs Q&A) :

```
def present(self, slide_content: str, language: str = "fr",
            chapter_title: str = "", student_level: str = "", domain: str = None,
            previous_concept: str = "", previous_narration_summary: str = "") -> str:
    # 1. Cache check (md5(content+lang+level+domain+ch+sec) → narration text on disk)
    key = _narration_key(slide_content, language, student_level, domain, chapter_title, "")
    cached = _narration_cache_read(key)
    if cached:
        log.info(f"■ Narration cache HIT (key={key[:8]}, {len(cached)} chars)")
        return cached

    # 2. Build presentation prompt
    system_content = get_presentation_prompt(domain, language, chapter_title, student_level)

    # 3. Continuity bridge (si previous_concept fourni)
    user_content = slide_content
    if previous_concept and previous_narration_summary:
        user_content = (
            f"■ CONTINUITÉ : tu viens de couvrir '{previous_concept}'.\n"
            f"Résumé de la précédente narration : {previous_narration_summary}\n\n"
            f"SLIDE ACTUELLE :\n{slide_content}"
        )

    messages = [{"role": "system", "content": system_content}, {"role": "user", "content": user_content}]

    # 4. Same fallback chain : OpenAI/Groq → Ollama
    if self.client:
        try:
            response = self.client.chat.completions.create(
                model=self._client_model, messages=messages,
                temperature=0.5, max_tokens=600,
            )
            narration = self._clean_for_speech(response.choices[0].message.content)
            _narration_cache_write(key, narration) # cache 7j
            return narration
        except Exception:
            pass

    # 5. Ollama fallback ...
    return narration
```

5. ai/local_llm.py

Rôle : wrapper Ollama avec timing logs détaillés.

5.1 LocalLLMFallback.__init__

```
class LocalLLMFallback:
    def __init__(self, model="mistral", base_url="http://localhost:11434"):
        self.model = model
        self.base_url = base_url
        self.endpoint = f"{base_url}/api/generate"
```

```

self.available = self._check_availability()

def _check_availability(self) -> bool:
    try:
        response = requests.get(f"{self.base_url}/api/tags", timeout=None)
        if response.status_code == 200:
            models = response.json().get('models', [])
            model_names = [m.get('name', '').split(':')[0] for m in models]
            is_available = any(self.model in name for name in model_names)
            if is_available:
                log.info(f"■ Ollama actif - modèle '{self.model}' chargé")
            else:
                log.warning(f"■ Service Ollama actif, modèle '{self.model}' non présent")
            return is_available
        except Exception:
            log.warning(f"■ Ollama non connecté sur {self.base_url}")
            return False

```

5.2 generate() — async LLM call

```

async def generate(self, prompt, temperature=0.7, max_tokens=500) -> Optional[str]:
    if not self.available:
        return None

    import time as _ot
    try:
        _opts = {"temperature": temperature, "num_predict": max_tokens}
        try:
            from core.config import Config as _Cfg
            _n_threads = int(getattr(_Cfg, "OLLAMA_NUM_THREADS", 0) or 0)
            if _n_threads > 0:
                _opts["num_thread"] = _n_threads
        except Exception:
            pass
        payload = {"model": self.model, "prompt": prompt, "stream": False, "options": _opts}

        log.info(f"■ OLLAMA generate START | model=%s | prompt_chars=%d (~%d tokens) | temp=%.2f max_tokens=%d t=%s" % (
            self.model, len(prompt), len(prompt)//4, temperature, max_tokens,
            _opts.get("num_thread", "default"), self.endpoint))
        _t0 = _ot.time()

        response = requests.post(self.endpoint, json=payload, timeout=None)
        elapsed = _ot.time() - _t0

        if response.status_code == 200:
            result = response.json()
            generated_text = result.get('response', '').strip()
            # Ollama returns these stats when stream=false :
            eval_count = int(result.get("eval_count") or 0)
            eval_dur_ns = int(result.get("eval_duration") or 0)
            prompt_eval_count = int(result.get("prompt_eval_count") or 0)
            prompt_eval_dur_ns = int(result.get("prompt_eval_duration") or 0)
            load_dur_ns = int(result.get("load_duration") or 0)
            tokens_per_s = (eval_count / (eval_dur_ns / 1e9)) if eval_dur_ns > 0 else 0.0

            log.info(f"■ OLLAMA generate DONE | model=%s | prompt_tokens=%d eval_tokens=%d | %.0f tok/s | load=%s" % (
                self.model, prompt_eval_count, eval_count, tokens_per_s,
                load_dur_ns/1e6, prompt_eval_dur_ns/1e6, eval_dur_ns/1e6,
                elapsed, len(generated_text)))
            return generated_text or None
        except Exception as e:
            log.error(f"■ OLLAMA generate ERR | {e}")
            return None

```

Le log avec **prompt_tokens**, **eval_tokens**, **tok/s**, **load/prompt_eval/eval** nanoseconds → permet de diagnostiquer la latence d'Ollama (où passe le temps : load, prompt eval, generation).

6. handlers/auth.py

Rôle : helpers JWT + bcrypt + rate limiting + audit.

6.1 check_password_strength (lignes 45-72)

```
_COMMON_PASSWORDS = {
    "password", "12345678", "qwerty123", "admin123", "password1",
    "letmein", "iloveyou", "azerty123", "motdepasse", "12345abc",
}

def check_password_strength(password: str) -> None:
    pw_len = len(password) if password else 0
    has_alpha = bool(re.search(r"[a-zA-Z]", password)) if password else False
    has_digit = bool(re.search(r"\d", password)) if password else False
    is_common = (password or "").lower() in _COMMON_PASSWORDS
    log.info("■ password_strength | len=%d alpha=%s digit=%s common=%s",
            pw_len, has_alpha, has_digit, is_common)
    if pw_len < 8:
        raise PasswordStrengthError("Mot de passe trop court (minimum 8 caractères)")
    if is_common:
        raise PasswordStrengthError("Mot de passe trop commun, choisissez-en un autre")
    if not (has_alpha and has_digit):
        raise PasswordStrengthError("Le mot de passe doit contenir lettres et chiffres")
```

3 règles minimales : ≥8 chars, alpha+digit mélangés, pas dans liste de 10 mots de passe communs.

6.2 hash_password / verify_password (bcrypt rounds=12)

```
def hash_password(plain: str) -> str:
    t0 = time.perf_counter()
    h = bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt(rounds=12)).decode("utf-8")
    log.info("■ hash_password | bcrypt rounds=12 | took=%.0fms | hash_prefix=%s",
            (time.perf_counter() - t0) * 1000.0, h[:7])
    return h

def verify_password(plain: str, hashed: str) -> bool:
    if not plain or not hashed:
        return False
    try:
        t0 = time.perf_counter()
        ok = bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))
        log.info("■ verify_password | match=%s | bcrypt compare took=%.0fms",
                ok, (time.perf_counter() - t0) * 1000.0)
        return ok
    except Exception as exc:
        log.warning(f"■ verify_password ERROR: {exc}")
        return False
```

bcrypt rounds=12 → ~180-250ms par check sur CPU moderne. Volontairement lent pour ralentir le brute-force.

6.3 create_access_token / decode_access_token (JWT)

```
def create_access_token(student_id, email, account_level="student", expires_hours=None) -> str:
    hours = expires_hours or Config.JWT_EXPIRATION_HOURS
    exp = datetime.utcnow() + timedelta(hours=hours)
    payload = {
        "sub": student_id,
```

```

        "email": email,
        "account_level": account_level,
        "iat": datetime.utcnow(),
        "exp": exp,
    }
    token = jwt.encode(payload, Config.JWT_SECRET_KEY, algorithm=Config.JWT_ALGORITHM)
    log.info("■ JWT CREATE | sub=%s email=%s role=%s | algo=%s ttl=%dh exp=%s | token_prefix=%s...",
            str(student_id)[:8], email, account_level,
            Config.JWT_ALGORITHM, hours, exp.isoformat(), token[:20])
    return token

def decode_access_token(token: str) -> dict:
    try:
        claims = jwt.decode(token, Config.JWT_SECRET_KEY, algorithms=[Config.JWT_ALGORITHM])
        log.info("■ JWT DECODE OK | sub=%s email=%s role=%s exp=%s",
                str(claims.get("sub", ""))[:8], claims.get("email", ""),
                claims.get("account_level", ""),
                datetime.utcfromtimestamp(claims["exp"]).isoformat())
        return claims
    except jwt.ExpiredSignatureError:
        log.warning("■ JWT DECODE FAIL | reason=expired_signature")
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Token expired")
    except jwt.InvalidTokenError as exc:
        log.warning("■ JWT DECODE FAIL | reason=invalid_token (%s)", exc)
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, f"Invalid token: {exc}")

```

HS256 = HMAC SHA-256 avec clé symétrique `Config.JWT_SECRET_KEY`. Simple et suffisant pour single-tenant.

6.4 Rate limiting Redis sliding window

```

async def check_login_rate_limit(identifiant, max_attempts=5, window_seconds=300):
    try:
        from pedagogy.dialogue import get_redis
        r = await get_redis()
        key = f"login:fails:{identifiant.lower()}"
        n = int(await r.get(key) or 0)
        allowed = n < max_attempts
        remaining = max(0, max_attempts - n)
        log.info("■ rate_limit CHECK | id=%s | failures=%d/%d window=%ds | allowed=%s remaining=%d",
                identifiant.lower(), n, max_attempts, window_seconds, allowed, remaining)
        if not allowed:
            return (False, 0)
        return (True, remaining)
    except Exception as exc:
        log.warning(f"■ rate_limit CHECK skipped (Redis down?): {exc} - failing OPEN")
        return (True, max_attempts)

async def record_login_failure(identifiant, window_seconds=300) -> int:
    try:
        from pedagogy.dialogue import get_redis
        r = await get_redis()
        key = f"login:fails:{identifiant.lower()}"
        n = await r.incr(key)
        if n == 1:
            await r.expire(key, window_seconds) # set TTL on first fail
        return int(n)
    except Exception as exc:
        return 0

```

Sliding window simple : INCR + EXPIRE 5 min sur premier fail. Au 6e fail dans la fenêtre → 429.

Si Redis down → **fail OPEN** (= permettre les logins) : choix défensif vs DoS, mais ça désactive le rate limit. Tradeoff explicite.

6.5 audit_log (CSV append-only)

```
_AUDIT_PATH = os.path.join(Config.LOGS_DIR, "sec_audit.csv")

def audit_log(event, actor="", target="", request=None, outcome="success", details=""):
    try:
        _ensure_audit_file()
        ip = (request.client.host if request and request.client else "")
        ua = (request.headers.get("user-agent", "") or "")[:120]
        with open(_AUDIT_PATH, "a", newline="", encoding="utf-8") as f:
            csv.writer(f).writerow([
                datetime.utcnow().isoformat(), event, actor[:100], target[:100],
                ip, ua, outcome, details[:200],
            ])
        log.info("■ AUDIT | event=%s actor=%s target=%s ip=%s outcome=%s | %s",
                event, actor[:40], target[:60], ip, outcome, details[:80])
    except Exception as exc:
        log.warning(f"■ audit_log write FAILED: {exc}")
```

Append-only CSV avec colonnes : timestamp, event, actor, target, ip, ua, outcome, details.

Events typiques : `register`, `register_failed`, `login`, `login_failed`, `login_rate_limited`, `login_disabled`, `logout`, `password_change`, `role_change`.

7. routes/auth.py

Rôle : 4 endpoints REST `/auth/*`.

7.1 POST `/auth/register`

```
@router.post("/register", response_model=TokenOut)
async def register(payload: RegisterIn, request: Request, response: Response):
    log.info("■ REGISTER START | email=%s lang=%s level=%s | ip=%s", ...)

    # 1. Password strength check (raise PasswordStrengthError)
    try:
        check_password_strength(payload.password)
    except PasswordStrengthError as exc:
        audit_log("register_failed", target=payload.email, request=request,
                outcome="failure", details=str(exc))
        raise HTTPException(400, str(exc))

    # 2. Check email unique
    async with AsyncSessionLocal() as db:
        existing = (await db.execute(
            select(Student).where(Student.email == payload.email.lower())
        )).scalar_one_or_none()
        if existing is not None:
            raise HTTPException(409, "Email already registered")

    # 3. Validate level + language whitelist
    _allowed_levels = {"collège", "lycée", "université"}
    student_level = payload.student_level if payload.student_level in _allowed_levels else "lycée"
    _allowed_langs = {"fr", "en", "ar"}
    preferred_language = payload.preferred_language if payload.preferred_language in _allowed_langs else "fr"

    # 4. Create student
    student = Student(
        id=uuid.uuid4(),
```

```

        email=payload.email.lower(),
        password_hash=hash_password(payload.password),      # bcrypt slow
        first_name=payload.first_name,
        last_name=payload.last_name,
        preferred_language=preferred_language,
        student_level=student_level,
        account_level="student",
        is_active=1,
    )
    db.add(student)
    await db.commit()
    await db.refresh(student)

# 5. Issue JWT + set cookie
token = create_access_token(str(student.id), student.email, student.account_level)
response.set_cookie("smart_teacher_token", token,
                    httponly=True, samesite="lax", max_age=24 * 3600)
audit_log("register", actor=str(student.id), target=student.email, request=request)
return TokenOut(access_token=token, ...)

```

7.2 POST /auth/login

```

@router.post("/login", response_model=TokenOut)
async def login(payload: LoginIn, request: Request, response: Response):
    email_lower = payload.email.lower()

    # 1. Rate limit BEFORE password check (anti timing-based enumeration)
    allowed, remaining = await check_login_rate_limit(email_lower)
    if not allowed:
        audit_log("login_rate_limited", target=email_lower, request=request,
                 outcome="failure", details="too many attempts")
        raise HTTPException(429, "Trop de tentatives échouées. Réessaie dans 5 minutes.")

    # 2. Lookup + verify password
    async with AsyncSessionLocal() as db:
        student = (await db.execute(
            select(Student).where(Student.email == email_lower)
        )).scalar_one_or_none()
        # SAME error message for unknown email vs wrong password (anti enumeration)
        if student is None or not verify_password(payload.password, student.password_hash or ""):
            await record_login_failure(email_lower)
            raise HTTPException(401, "Invalid email or password")
        if not student.is_active:
            raise HTTPException(403, "Account disabled")

    # 3. Reset fail counter + issue JWT + set cookie
    await reset_login_failures(email_lower)
    token = create_access_token(str(student.id), student.email, student.account_level)
    response.set_cookie("smart_teacher_token", token, httponly=True, samesite="lax", max_age=24*3600)
    return TokenOut(...)

```

Ordre IMPORTANT : rate limit AVANT password check. Sinon un attaquant peut timer les checks pour distinguer "email inconnu" (rapide) vs "password incorrect" (lent à cause de bcrypt).

7.3 POST /auth/logout

```

@router.post("/logout")
async def logout(response: Response, request: Request):
    response.delete_cookie("smart_teacher_token")
    log.info("■ LOGOUT | cookie cleared | ip=%s", request.client.host)
    return {"status": "ok"}

```

7.4 GET /auth/me

```
@router.get("/me", response_model=MeOut)
async def me(user: dict = Depends(get_current_user)):
    async with AsyncSessionLocal() as db:
        student = (await db.execute(
            select(Student).where(Student.id == uuid.UUID(user["sub"])))
        ).scalar_one_or_none())
        if student is None:
            raise HTTPException(404, "Account not found")
    return MeOut(...)
```

`Depends(get_current_user)` extrait + vérifie le JWT. `user["sub"] = student_id`.

8. pedagogy/dialogue.py

Rôle : FSM dialogue + SessionContext + pause/resume + Redis I/O. ~700 lignes.

8.1 DialogState enum

```
class DialogState(str, Enum):
    IDLE = "IDLE" # aucune session active
    INDEXING = "INDEXING" # ingestion d'un cours en cours
    PRESENTING = "PRESENTING" # TTS narration en cours
    LISTENING = "LISTENING" # VAD actif, attente d'audio étudiant
    PROCESSING = "PROCESSING" # STT + RAG + LLM en cours
    RESPONDING = "RESPONDING" # TTS de la réponse
    WAITING = "WAITING" # pause utilisateur
    CLARIFICATION = "CLARIFICATION" # étudiant demande clarification
```

8.2 VALID_TRANSITIONS

```
VALID_TRANSITIONS: dict[DialogState, list[DialogState]] = {
    DialogState.IDLE: [INDEXING, PRESENTING, LISTENING],
    DialogState.INDEXING: [IDLE, PRESENTING],
    DialogState.PRESENTING: [LISTENING, WAITING, CLARIFICATION],
    DialogState.LISTENING: [PROCESSING, PRESENTING, CLARIFICATION],
    DialogState.PROCESSING: [RESPONDING, CLARIFICATION, IDLE, LISTENING, PRESENTING],
    DialogState.RESPONDING: [PRESENTING, WAITING, LISTENING, CLARIFICATION],
    DialogState.WAITING: [PRESENTING, LISTENING, PROCESSING, IDLE, CLARIFICATION],
    DialogState.CLARIFICATION: [RESPONDING, PRESENTING, LISTENING],
}
```

■ Note : **WAITING** → **PROCESSING** ajouté après bug fix (sinon student paused qui pose une question crashait avec « Transition invalide »).

8.3 SessionContext (dataclass complet)

```
@dataclass
class SessionContext:
    session_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    state: str = DialogState.IDLE.value
    language: str = "fr"
    student_level: str = "lycée"

    # Position dans le cours
    course_id: Optional[str] = None
    chapter_index: int = 0
```

```

section_index:    int = 0
char_position:   int = 0
last_narrated_idea_id: str = ""
student_id:      str = ""

# Course metadata
course_summary:  str = ""
course_analysis: dict = field(default_factory=dict)
last_slide_explained: str = ""
last_concept_explained: str = ""
last_narration_summary: str = ""

# Confusion
confusion_count:    int = 0
last_question_hash: str = ""
repeated_question_count: int = 0
last_confusion_score: float = 0.0
consecutive_clean_turns: int = 0

# Adaptive student baseline (Couche #1)
student_baseline: dict = field(default_factory=lambda: {
    "avg_speech_rate": 120.0,
    "avg_question_length": 8,
    "avg_questions_per_turn": 1.2,
    "hesitation_baseline": 1.0,
    "confusion_threshold_multiplier": 1.0,
    "turns_analyzed": 0,
})

# Pause/reprise
paused_state: dict = field(default_factory=lambda: {
    "is_paused": False,
    "slide_id": None,
    "char_offset": 0,
    "last_idea_id": "",
    "timestamp": 0.0,
    "presentation_text": "",
    "presentation_cursor": 0,
    "presentation_key": "",
    "slide_title": "",
    "slide_path": "",
    "slide_content": "",
    "presentation_text_len": 0,
})

interruptions:    int = 0
last_activity:    float = field(default_factory=time.time)

def to_json(self) -> str:
    return json.dumps(asdict(self), ensure_ascii=False, default=str)

@classmethod
def from_json(cls, raw: str) -> "SessionContext":
    return cls(**json.loads(raw))

```

Sérialisé en JSON → Redis. Pattern : pure data class, pas de méthode métier.

8.4 _save / _load (Redis I/O)

```

async def _save(self, ctx: SessionContext) -> None:
    r = await get_redis()
    key = f"session:{ctx.session_id}"
    payload = ctx.to_json()
    _t0 = time.time()
    await r.setex(key, SESSION_TTL, payload) # SESSION_TTL = 3600s
    log.debug("■ REDIS SETEX | key=%s ttl=%ds size=%dB state=%s took=%.1fms",
              key, SESSION_TTL, len(payload), ctx.state,
              (time.time() - _t0) * 1000)

```

```

async def _load(self, session_id: str) -> Optional[SessionContext]:
    r = await get_redis()
    key = f"session:{session_id}"
    _t0 = time.time()
    data = await r.get(key)
    elapsed = (time.time() - _t0) * 1000
    if not data:
        log.debug("■ REDIS GET MISS | key=%s took=%.1fms", key, elapsed)
        return None
    log.debug("■ REDIS GET HIT | key=%s size=%dB took=%.1fms", key, len(data), elapsed)
    return SessionContext.from_json(data)

```

8.5 transition()

```

async def transition(self, session_id: str, to_state: DialogState) -> Optional[SessionContext]:
    ctx = await self._load(session_id)
    if not ctx:
        return None
    from_state = DialogState(ctx.state)
    if to_state not in VALID_TRANSITIONS.get(from_state, []):
        log.error("■ [%s] Transition invalide : %s → %s",
            session_id[:8], from_state.value, to_state.value)
        return None
    ctx.state = to_state.value
    ctx.last_activity = time.time()
    await self._save(ctx)
    log.info("■ [%s] %s → %s", session_id[:8], from_state.value, to_state.value)
    return ctx

```

Transition validée par le dict statique. Si invalide → log error + return None (pas d'exception).

8.6 pause_session() — sauve le pause cursor + slide context

```

async def pause_session(self, session_id, slide_id=None, char_offset=0,
    presentation_text=None, presentation_cursor=None,
    presentation_key=None, slide_title=None,
    last_idea_id=None, slide_path=None, slide_content=None):
    ctx = await self._load(session_id)
    if not ctx:
        return None

    effective_idea_id = last_idea_id or ctx.last_narrated_idea_id or ""

    # Compare slide vs précédent – si MÊME slide, garder l'ancien path/content
    prev_state = ctx.paused_state or {}
    prev_slide_id = str(prev_state.get("slide_id") or prev_state.get("presentation_key") or "")
    same_slide_as_prev = bool(prev_slide_id) and prev_slide_id == str(slide_id or "")

    effective_slide_path = (
        slide_path if slide_path is not None
        else (prev_state.get("slide_path", "") if same_slide_as_prev else "")
    ) or ""
    effective_slide_content = (
        slide_content if slide_content is not None
        else (prev_state.get("slide_content", "") if same_slide_as_prev else "")
    ) or ""

    # Last-resort lookup pour nouvelle slide
    if (not effective_slide_path or not effective_slide_content) and ctx.course_id and slide_id:
        try:
            from services.course_slides import load_course_slide_context
            parts = str(slide_id).split(":")
            if len(parts) >= 3:
                ch_idx = int(parts[1])

```

```

        sec_idx = int(parts[2])
        slide_ctx = await load_course_slide_context(ctx.course_id, ch_idx, sec_idx)
        if slide_ctx:
            if not effective_slide_path:
                effective_slide_path = slide_ctx.get("slide_path") or ""
            if not effective_slide_content:
                effective_slide_content = slide_ctx.get("content") or ""
            log.info("■ pause_session slide_lookup | course=%s slide=%s | path=%s content_len=%d (refre
                str(ctx.course_id)[:8], slide_id,
                "yes" if effective_slide_path else "no",
                len(effective_slide_content or ""))
        except Exception as exc:
            log.debug(f"slide_path lookup skipped: {exc}")

    _pause_ts = time.time()
    _eff_cursor = max(0, presentation_cursor if presentation_cursor is not None else char_offset)
    _text_len = len(presentation_text or "")
    _progress = (_eff_cursor / _text_len * 100.0) if _text_len else 0.0

    ctx.paused_state = {
        "is_paused": True,
        "slide_id": slide_id,
        "char_offset": char_offset,
        "last_idea_id": effective_idea_id,
        "timestamp": _pause_ts,
        "presentation_text": presentation_text or "",
        "presentation_cursor": _eff_cursor,
        "presentation_key": presentation_key or slide_id or "",
        "slide_title": slide_title or "",
        "slide_path": effective_slide_path,
        "slide_content": effective_slide_content,
        "presentation_text_len": _text_len,
    }
    ctx.char_position = char_offset
    if effective_idea_id:
        ctx.last_narrated_idea_id = effective_idea_id
    ctx.interruptions += 1
    ctx.state = DialogState.WAITING.value
    ctx.last_activity = time.time()
    await self._save(ctx)

    if slide_id:
        await self.save_presentation_snapshot(...)

    log.info("■■■ [%s] PAUSE_SESSION | slide=%s | cursor=%d/%d (%.1f%%) | char_offset=%d | idea=%s | timestamp=
        session_id[:8], slide_id, _eff_cursor, _text_len, _progress, char_offset,
        effective_idea_id or '-',
        _dt.utctimestamp(_pause_ts).isoformat(), _pause_ts,
        ctx.interruptions)
    return ctx

```

L'invariance critique : quand l'étudiant change de slide entre 2 pauses, `slide_path` et `slide_content` doivent être ré-chargés (sinon le Q&A handler reçoit le contenu de l'ancienne slide → bug observable).

8.7 resume_session()

```

async def resume_session(self, session_id) -> Optional[SessionContext]:
    ctx = await self._load(session_id)
    if not ctx:
        return None
    if not ctx.paused_state.get("is_paused"):
        log.warning("[%s] ■■ RESUME_SESSION SKIP | session is not paused", session_id[:8])
        return ctx

    offset = ctx.paused_state.get("presentation_cursor",
                                   ctx.paused_state.get("char_offset", 0))

    _now = time.time()
    _pause_ts = ctx.paused_state.get("timestamp")

```

```

try:
    _pause_dur = max(0.0, _now - float(_pause_ts)) if _pause_ts else 0.0
except (TypeError, ValueError):
    _pause_dur = 0.0

_text_len = ctx.paused_state.get("presentation_text_len", 0) or len(ctx.paused_state.get("presentation_text"))
_progress = (offset / _text_len * 100.0) if _text_len else 0.0
_slide_id = ctx.paused_state.get("slide_id") or ctx.paused_state.get("presentation_key") or "?"

ctx.paused_state["is_paused"] = False
ctx.paused_state["char_offset"] = offset
ctx.paused_state["presentation_cursor"] = offset
ctx.char_position = offset
ctx.state = DialogState.PRESENTING.value
ctx.last_activity = _now
await self._save(ctx)

log.info("■■ [%(s)] RESUME_SESSION | slide=%(s) | cursor=%d/%d (%.1f%%) | pause_duration=%(f)s | resume_at=%(s)"
        % (_now, session_id[:8], _slide_id, offset, _text_len, _progress, _pause_dur,
          _dt.utcnow().isoformat()))
return ctx

```

`_pause_dur = now - timestamp` → durée de pause exacte (pour ResumeIntelligence).

8.8 save_presentation_snapshot / load_presentation_snapshot (Redis)

```

async def save_presentation_snapshot(self, session_id, slide_id, presentation_text,
                                     presentation_cursor=0, slide_title=""):
    if not slide_id:
        return
    payload = {
        "session_id": session_id,
        "slide_id": slide_id,
        "presentation_text": presentation_text or "",
        "presentation_cursor": max(0, presentation_cursor),
        "slide_title": slide_title or "",
        "presentation_text_len": len(presentation_text or ""),
        "updated_at": time.time(),
    }
    r = await get_redis()
    key = self._presentation_snapshot_key(session_id, slide_id)
    body = json.dumps(payload, ensure_ascii=False)
    _t0 = time.time()
    await r.setex(key, PRESENTATION_SNAPSHOT_TTL, body)
    log.info("■ REDIS SETEX snapshot | key=presentation:snapshot:%s:%s | size=%dB ttl=%ds | text_len=%d cursor=%d"
            % (session_id[:8], slide_id, len(body), PRESENTATION_SNAPSHOT_TTL,
              len(presentation_text or ""), presentation_cursor,
              (time.time() - _t0) * 1000))

```

Clé : `presentation:snapshot:{session_id}:{course_id}:{chapter_idx}:{section_idx}`. TTL 1h.

8.9 save_tts_phrase_cache / load_tts_phrase_cache (Redis)

```

async def save_tts_phrase_cache(self, text, audio_bytes, *, language, rate, provider, voice_name, mime="audio/mpeg"):
    if not text or not audio_bytes:
        return ""
    payload = {
        "text": text,
        "audio_b64": base64.b64encode(audio_bytes).decode("ascii"),
        "mime": mime or "audio/mpeg",
        "language": language,
    }

```

```

        "rate": rate,
        "provider": provider,
        "voice_name": voice_name,
        "metadata": metadata or {},
        "created_at": time.time(),
    }
    cache_key = self._tts_cache_key(text, language, rate, provider, voice_name)
    body = json.dumps(payload, ensure_ascii=False)
    r = await get_redis()
    await r.setex(cache_key, TTS_CACHE_TTL, body) # TTS_CACHE_TTL = 24h
    log.info("■ REDIS SETEX tts_phrase | key=%s... | size=%dB ttl=%ds | audio=%dB chars=%d %s/%s %s %s | took=%s" % (
        cache_key[:30], len(body), TTS_CACHE_TTL,
        len(audio_bytes), len(text), provider, voice_name, language, rate,
        (time.time() - _t0) * 1000))
    return cache_key

```

Cache key calculé par md5 :

```

@staticmethod
def _tts_cache_key(text, language, rate, provider, voice_name) -> str:
    normalized = "|".join([
        provider.strip().lower(),
        voice_name.strip().lower(),
        language.strip().lower(),
        rate.strip().lower(),
        text.strip(),
    ])
    digest = hashlib.sha1(normalized.encode("utf-8")).hexdigest()
    return f"presentation:tts:{digest}"

```

→ Identical (texte, voice, rate, lang, provider) → même clé → cache hit.

9. pedagogy/mastery_repo.py

Rôle : repository sans état pour **student_mastery** (Beta-Binomial + Laplace).

9.1 _laplace_score (formule, lignes 87-95)

```

def _laplace_score(attempts: int, confusions: int) -> float:
    """Posterior mean under Beta(1,1) prior – Laplace rule of succession.

     $\theta = (correct + 1) / (attempts + 2)$ 

    Returns 0.5 (the uniform prior) when ``attempts == 0``.
    """
    correct = max(0, attempts - confusions)
    return (correct + 1) / (attempts + 2)

```

Worked examples :

attempts	confusions	correct	$(c+1)/(a+2)$	θ
0	0	0	1/2	0.500
5	0	5	6/7	0.857
5	1	4	5/7	0.714
10	5	5	6/12	0.500

9.2 get_score / get_scores_bulk

```
@staticmethod
async def get_score(student_id, course_id, idea_id: str) -> float:
    sid = _coerce_uuid(student_id)
    cid = _coerce_uuid(course_id)
    if not sid or not idea_id:
        return 0.5
    try:
        async with AsyncSessionLocal() as db:
            stmt = select(StudentMastery.attempts, StudentMastery.confusions).where(
                StudentMastery.student_id == sid,
                StudentMastery.course_id == cid,
                StudentMastery.idea_id == idea_id,
            )
            row = (await db.execute(stmt)).first()
            if row is None:
                return 0.5 # cold start
            attempts, confusions = int(row[0] or 0), int(row[1] or 0)
            return _laplace_score(attempts, confusions)
    except Exception:
        return 0.5 # failsafe
```

Cold start = 0.5 partout (uniform prior).

`get_scores_bulk(student_id, course_id, idea_ids: list[str])` → SELECT IN clause → dict
`{idea_id: score}`.

9.3 update_mastery

```
@staticmethod
async def update(student_id, course_id, idea_id, is_confusion: bool, label: Optional[str] = None) -> Optional[float]:
    sid = _coerce_uuid(student_id)
    cid = _coerce_uuid(course_id)
    if not sid or not idea_id:
        return None
    try:
        async with AsyncSessionLocal() as db:
            stmt = select(StudentMastery).where(
                StudentMastery.student_id == sid,
                StudentMastery.course_id == cid,
                StudentMastery.idea_id == idea_id,
            )
            existing = (await db.execute(stmt)).scalar_one_or_none()
            if existing is None:
                # New entry – first interaction with this idea
                _correct = 0 if is_confusion else 1
                attempts = 1
                confusions = 1 if is_confusion else 0
                new_score = _laplace_score(attempts, confusions)
                new_row = StudentMastery(
                    student_id=sid, course_id=cid, idea_id=idea_id,
                    attempts=attempts, confusions=confusions, score=new_score,
                    last_seen_at=datetime.utcnow(),
                )
                db.add(new_row)
                await db.commit()
                log.info("■ mastery UPDATE | idea=%s | %s | correct=%d/%d | Beta(1,1)+Laplace : (%d+1)/(%d+2) = %f",
                    str(idea_id)[:30], "CONFUSION" if is_confusion else "CLEAN",
                    _correct, attempts, _correct, attempts, new_score, new_score - 0.5)
                return new_score
            else:
                # Existing – update counters
                old_score = float(existing.score or 0.5)
                existing.attempts += 1
                if is_confusion:
```

```

        existing.confusions = (existing.confusions or 0) + 1
        _correct = max(0, existing.attempts - (existing.confusions or 0))
        new_score = _laplace_score(existing.attempts, existing.confusions or 0)
        existing.score = new_score
        existing.last_seen_at = datetime.utcnow()
        await db.commit()
        log.info("■ mastery UPDATE | idea=%s | %s | correct=%d/%d (confusions=%d) | Beta(1,1)+Laplace :
                  str(idea_id)[:30], "CONFUSION" if is_confusion else "CLEAN",
                  _correct, existing.attempts, existing.confusions or 0,
                  _correct, existing.attempts, new_score, old_score, new_score - old_score)
    return new_score
except Exception:
    return None

```

Pattern : SELECT puis UPDATE/INSERT. Pas d'UPSERT atomic (suffisant pour notre charge).

10. pedagogy/review_scheduler.py (FSRS)

Rôle : intégration FSRS pour spaced repetition.

10.1 Helpers Card serialization

```

def _serialize_card(card) -> dict[str, Any]:
    """fsrs Card → JSON dict via to_dict() if available, sinon manual."""
    try:
        return card.to_dict()
    except Exception:
        return {
            "due": card.due.isoformat() if card.due else None,
            "stability": getattr(card, "stability", 0.0),
            "difficulty": getattr(card, "difficulty", 0.5),
            "state": str(getattr(card, "state", "learning")),
            "step": getattr(card, "step", 0),
        }

```

```

def _deserialize_card(data: dict | None):
    from fsrs import Card
    if not data or not isinstance(data, dict):
        return Card() # nouveau, prior par défaut
    try:
        return Card.from_dict(data)
    except Exception:
        return Card()

```

`fsrs.Card` est l'objet stateful du FSRS. Sérialisé en JSONB dans `review_queue.fsrs_state` (Postgres column).

10.2 update_after_practice (méthode principale)

```

@staticmethod
async def update_after_practice(student_id, concept_name: str, rating_str: str) -> Optional[datetime]:
    """Hook après chaque practice/Q&A. Retourne datetime du prochain rappel."""
    from fsrs import Rating
    sid = _coerce_uuid(student_id)
    if not sid or not concept_name:
        return None

    rating_map = {

```

```

    "again": Rating.Again,      # = 1, fausse réponse
    "hard":  Rating.Hard,      # = 2, correct mais avec hints
    "good":  Rating.Good,      # = 3, correct normal
    "easy":  Rating.Easy,      # = 4, parfait instant
}
rating = rating_map.get(rating_str.lower(), Rating.Good)

try:
    async with AsyncSessionLocal() as db:
        row = (await db.execute(
            select(ReviewQueue).where(
                ReviewQueue.student_id == sid,
                ReviewQueue.concept_name == concept_name,
            )
        )).scalar_one_or_none()

        scheduler = _get_scheduler()          # FSRS Scheduler() default
        card = _deserialize_card(row.fsr_state if row else None)

        # FSRS review : mise à jour de la Card + log
        new_card, _log = scheduler.review_card(card, rating)
        next_due = getattr(new_card, "due", None)

        serialized = _serialize_card(new_card)

        if row is None:
            db.add(ReviewQueue(
                student_id=sid, concept_name=concept_name,
                fsrs_state=serialized,
                due=next_due or datetime.utcnow(),
                last_review=datetime.utcnow(),
                review_count=1,
                lapse_count=1 if rating == Rating.Again else 0,
                state=str(getattr(new_card, "state", "learning")),
                stability=float(getattr(new_card, "stability", 0.0)),
                difficulty=float(getattr(new_card, "difficulty", 0.5)),
            ))
        else:
            row.fsr_state = serialized
            row.due = next_due or datetime.utcnow()
            row.last_review = datetime.utcnow()
            row.review_count = (row.review_count or 0) + 1
            if rating == Rating.Again:
                row.lapse_count = (row.lapse_count or 0) + 1
            row.state = str(getattr(new_card, "state", row.state))
            row.stability = float(getattr(new_card, "stability", row.stability))
            row.difficulty = float(getattr(new_card, "difficulty", row.difficulty))

        await db.commit()
        _interval_days = ((next_due - datetime.utcnow()).total_seconds() / 86400.0
                           if next_due else 0.0)
        log.info("■ FSRS UPDATE | concept='%s' rating=%s | stability=%.2f difficulty=%.2f state=%s | review
                  concept_name[:40], rating_str,
                  float(getattr(new_card, "stability", 0.0)),
                  float(getattr(new_card, "difficulty", 0.5)),
                  str(getattr(new_card, "state", "?")),
                  ..., next_due.isoformat() if next_due else "none",
                  _interval_days)

        return next_due
except Exception as exc:
    log.warning(f"update_after_practice failed: {exc}")
    return None

```

10.3 list_due

```

@staticmethod
async def list_due(student_id, course_id=None, limit: int = 20) -> list[dict[str, Any]]:
    sid = _coerce_uuid(student_id)
    if not sid:

```

```

    return []
try:
    async with AsyncSessionLocal() as db:
        rows = (await db.execute(
            select(ReviewQueue)
            .where(ReviewQueue.student_id == sid,
                ReviewQueue.due <= datetime.utcnow())
            .order_by(ReviewQueue.due.asc())
            .limit(limit * 3)          # over-fetch, filter par course après
        )).scalars().all()

    # Resolve concept names + filter by course via KG (in-memory)
    try:
        from deps import get_rag
        from pedagogy.knowledge_graph import get_or_build
        kg = get_or_build(get_rag())
    except Exception:
        kg = None

    course_id_str = str(course_id) if course_id else None
    out = []
    for rq in rows:
        concept_label = rq.concept_name
        concept_display = rq.concept_name
        concept_course = None
        if kg is not None:
            ci = kg.get_concept(rq.concept_name)
            if ci:
                concept_label = ci.name
                concept_display = ci.canonical_name or ci.display_name or ci.name
                concept_course = ci.course_id
        if course_id_str and concept_course and concept_course != course_id_str:
            continue
        out.append({
            "concept_name":    concept_label,
            "concept_display": concept_display,
            "due":             rq.due.isoformat() if rq.due else None,
            "state":          rq.state,
            "stability":       round(rq.stability or 0.0, 2),
            "difficulty":      round(rq.difficulty or 0.5, 2),
            "review_count":    rq.review_count,
            "lapse_count":     rq.lapse_count,
        })
        if len(out) >= limit:
            break
    return out
except Exception:
    return []

```

Stage 3 design : **concept_id** (UUID FK → concept_kg) → **concept_name** (string = ConceptInfo.name dans le KG). Plus de FK strictes — on résout via le KG in-memory.

Continued in next sections — see Pages following...

11. pedagogy/engagement.py

Rôle : score d'engagement multi-signaux.

11.1 Constants & weights (lignes 50-65)

```

W_RECENCY      = Config.ENGAGEMENT_W_RECENCY      # 0.30
W_QUESTION     = Config.ENGAGEMENT_W_QUESTION     # 0.25
W_CONFUSION    = Config.ENGAGEMENT_W_CONFUSION    # 0.20
W_PASSIVE      = Config.ENGAGEMENT_W_PASSIVE      # 0.15
W_INTERRUPT    = Config.ENGAGEMENT_W_INTERRUPT    # 0.10

# Fail-fast assert
assert abs((W_RECENCY + W_QUESTION + W_CONFUSION + W_PASSIVE + W_INTERRUPT) - 1.0) < 1e-6, \
    "engagement weights must sum to 1.0 – check ENGAGEMENT_W_* env vars"

DISENGAGED_BELOW = Config.ENGAGEMENT_DISENGAGED_BELOW # 0.30
ENGAGED_ABOVE    = Config.ENGAGEMENT_ENGAGED_ABOVE   # 0.65

```

L'assert garantit que si quelqu'un override les poids via env vars sans rebalancer, le boot crash → safe.

11.2 _recency_score (linear decay)

```

def _recency_score(seconds_since_last: Optional[float]) -> float:
    if seconds_since_last is None:
        return 0.5 # neutre si manquant
    if seconds_since_last < Config.ENGAGEMENT_RECENT_FULL_S: # < 30s
        return 1.0
    if seconds_since_last > Config.ENGAGEMENT_FULLY_STALE_S: # > 600s
        return 0.0
    # Linear decay between RECENT_FULL_S and FULLY_STALE_S
    span = Config.ENGAGEMENT_FULLY_STALE_S - Config.ENGAGEMENT_RECENT_FULL_S
    return (Config.ENGAGEMENT_FULLY_STALE_S - seconds_since_last) / span

```

11.3 _question_score (Goldilocks zone)

```

def _question_score(questions_in_session: int, session_age_s: float) -> float:
    """Question rate (per minute) : 0.5 q/min ≈ optimal engagement.
    Source : Fredricks et al. 2004 – Behavioral / Cognitive engagement.
    """
    session_age_min = max(0.5, session_age_s / 60)
    q_per_min = questions_in_session / session_age_min
    if q_per_min == 0.0:
        return 0.3 # silence ambigu – peut être mastery OU disengagement
    if q_per_min < 0.5:
        return 0.5 + q_per_min # ramp 0.5 → 1.0 between 0 and 0.5
    if q_per_min < 2.0:
        return 1.0 - (q_per_min - 0.5) * 0.2 # decay 1.0 → 0.7
    if q_per_min < 4.0:
        return 0.7 - (q_per_min - 2.0) * 0.1 # decay 0.7 → 0.5
    return 0.5 # > 4 q/min = struggling

```

11.4 _confusion_score, _passive_score, _interrupt_score

```

def _confusion_score(confusions_in_session: int) -> float:
    if confusions_in_session == 0: return 1.0
    if confusions_in_session == 1: return 0.7
    if confusions_in_session == 2: return 0.4 # = ENGAGEMENT_CONFUSION_OPTIMAL
    if confusions_in_session == 3: return 0.3
    return 0.2 # ≥4

def _passive_score(consecutive_passive_slides: int) -> float:
    if consecutive_passive_slides == 0: return 1.0
    if consecutive_passive_slides == 1: return 0.85
    if consecutive_passive_slides == 2: return 0.6
    if consecutive_passive_slides == 3: return 0.45
    if consecutive_passive_slides <= 5: return 0.3
    return 0.1 # ≥10

```

```
def _interrupt_score(latency_ms: Optional[float]) -> float:
    if latency_ms is None:
        return 0.5
    fast = Config.ENGAGEMENT_INTERRUPT_FAST_MS # 200
    slow = Config.ENGAGEMENT_INTERRUPT_SLOW_MS # 2000
    if latency_ms <= fast: return 1.0
    if latency_ms >= slow: return 0.0
    # Linear decay
    return (slow - latency_ms) / (slow - fast)
```

11.5 compute_engagement (assemblage final)

```
def compute_engagement(signals: EngagementSignals) -> EngagementResult:
    rec = _recency_score(signals.seconds_since_last_interaction)
    qst = _question_score(signals.questions_in_session, signals.session_age_s)
    cfn = _confusion_score(signals.confusions_in_session)
    psv = _passive_score(signals.consecutive_passive_slides)
    itr = _interrupt_score(signals.last_interrupt_latency_ms)

    score = (
        W_RECENCY * rec +
        W_QUESTION * qst +
        W_CONFUSION * cfn +
        W_PASSIVE * psv +
        W_INTERRUPT * itr
    )
    score = round(max(0.0, min(1.0, score)), 3)

    if score < DISENGAGED_BELOW: label = "disengaged"
    elif score < ENGAGED_ABOVE: label = "neutral"
    else: label = "engaged"

    reason = (f"score={score:.2f} ({label}) | "
              f"recency={rec:.2f} questions={qst:.2f} confusion={cfn:.2f} "
              f"passive={psv:.2f} interrupt={itr:.2f}")
    return EngagementResult(score=score, label=label, reason=reason)
```

11.6 signals_from_ctx (helper)

```
def signals_from_ctx(ctx, now_ts=None) -> EngagementSignals:
    """Convenience : extract signals from a SessionContext."""
    now_ts = now_ts or time.time()
    return EngagementSignals(
        seconds_since_last_interaction = now_ts - (ctx.last_activity or now_ts),
        questions_in_session           = getattr(ctx, "questions_in_session", 0),
        session_age_s                  = now_ts - (ctx.created_at or now_ts),
        confusions_in_session          = ctx.confusion_count,
        consecutive_passive_slides     = getattr(ctx, "consecutive_passive_slides", 0),
        last_interrupt_latency_ms      = getattr(ctx, "last_interrupt_latency_ms", None),
    )
```

12. pedagogy/student_knowledge.py

Rôle : aggregated knowledge snapshot per student (mean of ideas → concepts).

12.1 Thresholds (sourced from Config)

```
STRONG_THRESHOLD = Config.KNOWLEDGE_STRONG_THRESHOLD # 0.7
```

```
WEAK_THRESHOLD = Config.KNOWLEDGE_WEAK_THRESHOLD # 0.4
CONFUSED_THRESHOLD = Config.KNOWLEDGE_CONFUSED_THRESHOLD # 0.3
MAX_LISTED_CONCEPTS = Config.KNOWLEDGE_MAX_LISTED_CONCEPTS # 8
MAX_RECENT_CONFUSIONS = 5
COLD_START_ATTEMPTS = Config.KNOWLEDGE_COLD_START_ATTEMPTS # 5
```

12.2 StudentKnowledgeSnapshot dataclass

```
@dataclass
class StudentKnowledgeSnapshot:
    student_id: str
    course_id: str
    mastery_by_concept: dict[str, float] = field(default_factory=dict)
    strong_concepts: list[str] = field(default_factory=list)
    weak_concepts: list[str] = field(default_factory=list)
    never_seen_concepts: list[str] = field(default_factory=list)
    recently_confused: list[str] = field(default_factory=list)
    total_attempts: int = 0

    @property
    def is_cold_start(self) -> bool:
        """True if attempts <= COLD_START_ATTEMPTS. Adaptive prompts
        should NOT claim mastery in this regime – too little data."""
        return self.total_attempts <= COLD_START_ATTEMPTS
```

12.3 build_snapshot

```
async def build_snapshot(student_id, course_id, *, kg=None) -> StudentKnowledgeSnapshot:
    snap = StudentKnowledgeSnapshot(
        student_id=str(student_id or ""),
        course_id=str(course_id or ""),
    )
    if not student_id or not course_id:
        return snap

    # 1. Get course concepts from KG
    if kg is None:
        try:
            from deps import get_rag
            from pedagogy.knowledge_graph import get_or_build
            kg = get_or_build(get_rag())
        except Exception:
            return snap

    try:
        concepts = list(kg.list_concepts(course_id))
    except Exception:
        return snap

    if not concepts:
        return snap

    # 2. Bulk-fetch mastery for all ideas
    all_idea_ids: list[str] = []
    concept_ideas: dict[str, list[str]] = {}
    for c in concepts:
        ids = list(getattr(c, "idea_ids", set()) or set())
        concept_ideas[c.name] = ids
        all_idea_ids.extend(ids)

    if not all_idea_ids:
        return snap

    try:
        from pedagogy.mastery_repo import MasteryRepo
        idea_scores = await MasteryRepo.get_scores_bulk(student_id, course_id, all_idea_ids)
```

```

except Exception:
    idea_scores = {}

# 3. Aggregate idea-level scores → concept-level (mean)
for c in concepts:
    ids = concept_ideas.get(c.name, [])
    scored = [idea_scores[i] for i in ids if i in idea_scores]
    if scored:
        mean_score = round(sum(scored) / len(scored), 3)
        snap.mastery_by_concept[c.name] = mean_score
        log.info("■ snapshot AGG | concept=%-30s | mean(%s) = %.3f / %d ideas",
                  c.name[:30],
                  ", ".join(f"{s:.2f}" for s in scored[:5]) + ("..." if len(scored) > 5 else ""),
                  mean_score, len(scored))

snap.total_attempts = len(idea_scores)

# 4. Classify by bucket
sorted_by_score = sorted(snap.mastery_by_concept.items(), key=lambda kv: kv[1], reverse=True)
snap.strong_concepts = [
    name for name, score in sorted_by_score
    if score >= STRONG_THRESHOLD
][:MAX_LISTED_CONCEPTS]
snap.weak_concepts = [
    name for name, score in sorted(
        ((n, s) for n, s in snap.mastery_by_concept.items() if s < WEAK_THRESHOLD),
        key=lambda kv: kv[1],
    )
][:MAX_LISTED_CONCEPTS]

attempted_names = set(snap.mastery_by_concept.keys())
snap.never_seen_concepts = [
    c.name for c in concepts if c.name not in attempted_names
][:MAX_LISTED_CONCEPTS]

# 5. Recently confused (heuristique : mastery <= 0.3)
snap.recently_confused = [
    name for name, score in snap.mastery_by_concept.items()
    if score <= CONFUSED_THRESHOLD
][:MAX_RECENT_CONFUSIONS]

return snap

```

12.4 to_prompt_context (FR + EN)

```

def to_prompt_context(snap: StudentKnowledgeSnapshot, lang: str = "fr") -> str:
    """Format snapshot as a prompt block ready to inject into the Responder.
    Wrapped in <<INSTRUCTION_INTERNE>> markers so the LLM treats it as
    adaptation context, not content to discuss.

```

```

    Returns "" for cold-start snapshots.
    """

```

```

    if snap.is_cold_start:
        return ""

```

```

    if lang.startswith("fr"):
        opener = "<<INSTRUCTION_INTERNE_NE_PAS_MENTIONNER_DANS_LA_REPONSE>>"
        closer = "<<FIN_INSTRUCTION_INTERNE>>"
        header = "État pédagogique de l'étudiant (à respecter discrètement) :"
        lines = []
        if snap.strong_concepts:
            lines.append(f"- Concepts MAÎTRISÉS (ne pas re-définir, juste citer) : {'', '.join(snap.strong_concepts)}")
        if snap.weak_concepts:
            lines.append(f"- Concepts FAIBLES (reformuler simplement, ajouter un exemple) : {'', '.join(snap.weak_concepts)}")
        if snap.recently_confused:
            lines.append(f"- Récemment CONFUS sur (éviter le jargon, prendre le temps) : {'', '.join(snap.recently_confused)}")
        if snap.never_seen_concepts:
            lines.append(f"- Concepts JAMAIS VUS (présenter la définition complète si évoqués) : {'', '.join(snap.never_seen_concepts)}")
        if not lines:

```

```

        return ""
    return f"{opener}\n{header}\n" + "\n".join(lines) + f"\n{closer}\n\n"

# English version (similaire)
...

```

13. pedagogy/confusion/fusion.py

Rôle : fusion SIGHT (texte) + prosodie en score continu [0, 1].

13.1 Constants

```

W_TEXT      = 0.7          # SIGHT carries 70% of the decision
W_PROSODY   = 0.3          # prosody adds 30%

assert abs((W_TEXT + W_PROSODY) - 1.0) < 1e-6, \
    "confusion fusion weights must sum to 1.0"

# Score per number of prosody markers
_PROSODY_SCORE_BY_COUNT = {
    0: 0.0,
    1: 0.30,
    2: 0.70,
    3: 1.00,
}

```

13.2 score_from_prosody

```

def score_from_prosody(prosody: Optional[dict]) -> float:
    """Convert transcriber prosody output into a [0, 1] score.

    Uses COUNT of triggered markers (not their identity) – combining
    'slow_speech_rate' + 'frequent_hesitations' is stronger than any
    single marker, and per-marker weights would be unjustified
    without calibration data.
    """
    if not prosody or not isinstance(prosody, dict):
        return 0.0
    markers = prosody.get("markers", [])
    if not isinstance(markers, (list, tuple)):
        return 0.0
    n = min(3, len([m for m in markers if isinstance(m, str)]))
    return _PROSODY_SCORE_BY_COUNT.get(n, 1.0)

```

13.3 fuse_confusion_signals (la formule centrale)

```

def fuse_confusion_signals(signals: ConfusionSignals) -> FusedConfusion:
    contributors: list[str] = []

    text_score = float(signals.text_score) if signals.text_score is not None else None
    if text_score is not None:
        text_score = max(0.0, min(1.0, text_score))
        contributors.append("text(sight)")

    prosody_score = score_from_prosody(signals.prosody_dict)
    if signals.prosody_dict is not None:
        contributors.append("prosody")

```

```

if not contributors:
    return FusedConfusion(score=0.0, text_score=0.0, prosody_score=0.0,
                           contributors=[], reason="no signals available")

# Weighted sum with renormalisation when only one signal is present
if text_score is not None and signals.prosody_dict is not None:
    score = W_TEXT * text_score + W_PROSODY * prosody_score
elif text_score is not None:
    score = text_score # full weight on remaining signal
else:
    score = prosody_score

score = round(max(0.0, min(1.0, score)), 3)

reason_parts = []
if text_score is not None:
    reason_parts.append(f"sight={text_score:.2f}")
if signals.prosody_dict is not None:
    n_markers = len([m for m in signals.prosody_dict.get("markers", []) if isinstance(m, str)])
    reason_parts.append(f"prosody={prosody_score:.2f} ({n_markers} markers)")
reason = " | ".join(reason_parts) + f" | fused={score:.2f}"

return FusedConfusion(
    score=score, text_score=text_score or 0.0, prosody_score=prosody_score,
    contributors=contributors, reason=reason,
)

```

Renormalisation = quand un signal manque, on prend l'autre à 100% (pas de pénalisation pour absence).

14. pedagogy/confusion/detector.py

Rôle : SIGHT XLM-RoBERTa fine-tuned classifier. Singleton load.

14.1 ConfusionModel class

```

class ConfusionModel:
    def __init__(self, model_path: str = None, threshold: float = 0.6, max_length: int = 96, device: str = "cpu"):
        from transformers import XLMRobertaModel, XLMRobertaTokenizer
        import torch
        self.threshold = threshold
        self.max_length = max_length
        self.device = device

        # Load base model + custom head trained on SIGHT dataset
        self.tokenizer = XLMRobertaTokenizer.from_pretrained("xlm-roberta-base")
        self.base = XLMRobertaModel.from_pretrained("xlm-roberta-base").to(device)
        self.head = SIGHTHead().to(device) # custom Linear → sigmoid

        # Load trained weights
        ckpt = torch.load(model_path, map_location=device)
        self.head.load_state_dict(ckpt["head"])
        self.base.load_state_dict(ckpt["base"], strict=False)
        self.base.eval()
        self.head.eval()
        log.info("Loaded SIGHT confusion model from %s (model=xlm-roberta-base, threshold=%.3f, max_length=%d,
                  model_path, threshold, max_length, device)

```

14.2 predict()

```

def predict(self, text: str) -> tuple[bool, float]:

```

```

"""Returns (is_confused, prob_confused) where prob ∈ [0, 1]."""
import torch
inputs = self.tokenizer(text, max_length=self.max_length, truncation=True, padding="max_length", return_tensors="pt")
inputs = {k: v.to(self.device) for k, v in inputs.items()}

with torch.no_grad():
    outputs = self.base(**inputs)
    cls_emb = outputs.last_hidden_state[:, 0, :] # [CLS] token embedding
    logit = self.head(cls_emb)
    prob = torch.sigmoid(logit).item()

return prob >= self.threshold, prob

```

XLNet-RoBERTa-base = 270M params. CPU forward pass ~25-200ms par utterance après warm load.

14.3 Singleton

```

@lru_cache(maxsize=1)
def get_confusion_model() -> ConfusionModel:
    return ConfusionModel(model_path=Config.CONFUSION_MODEL_PATH,
                           threshold=0.6, max_length=96, device="cpu")

def predict_confusion(text: str) -> tuple[bool, float]:
    return get_confusion_model().predict(text)

```

15. pedagogy/personalization/profile.py

Rôle : student profile manager (Redis-backed, course-scoped).

15.1 StudentProfile dataclass (champs principaux)

```

@dataclass
class StudentProfile:
    student_id: str
    course_id: Optional[str] = None
    language: str = "fr"
    level: str = "lycée"
    style: str = "visual" # VARK dominant
    pace: str = "normal" # slow|normal|fast
    depth: str = "balanced" # brief|balanced|deep
    tone: str = "challenging" # challenging|encouraging|neutral
    preferences: dict = field(default_factory=dict)

    # VARK posterior Dirichlet
    learning_style_posterior: dict = field(default_factory=lambda: {
        "visual": 0.25, "verbal": 0.25,
        "kinesthetic": 0.25, "read_write": 0.25,
    })

    avg_response_time_s: float = 0.0
    confusion_rate: float = 0.0
    preferred_explanation_depth: str = "balanced"
    turns_analyzed: int = 0
    last_updated: float = field(default_factory=time.time)

```

15.2 ProfileManager (Redis I/O)

```

class ProfileManager:
    KEY_FMT = "profile:{student_id}:{course_id}" # course_id=_global pour cross-course

    async def get_or_create(self, session_id, language=None, level=None, course_id=None) -> StudentProfile:
        # session_id souvent = student_id (legacy)
        cid = str(course_id) if course_id else "_global"
        key = self.KEY_FMT.format(student_id=session_id, course_id=cid)
        r = await get_redis()
        raw = await r.get(key)
        if raw:
            try:
                return StudentProfile(**json.loads(raw))
            except Exception:
                pass
        # Create new
        profile = StudentProfile(
            student_id=str(session_id), course_id=cid,
            language=language or "fr",
            level=level or "lycée",
        )
        await self.save(profile, course_id=course_id)
        log.info("Created new profile %s/%s (lang=%s, level=%s)",
            str(session_id)[:8], cid[:8], profile.language, profile.level)
        return profile

    async def save(self, profile: StudentProfile, course_id=None) -> None:
        cid = str(course_id) if course_id else profile.course_id or "_global"
        key = self.KEY_FMT.format(student_id=profile.student_id, course_id=cid)
        r = await get_redis()
        profile.last_updated = time.time()
        body = json.dumps(asdict(profile), ensure_ascii=False)
        await r.set(key, body) # PERMANENT – pas de TTL (persistant cross-session)

```

Pas de TTL → profile persiste indéfiniment. Source de vérité = Redis (syncé manuellement avec Postgres `student_profiles` table pour audit trail).

15.3 Course-scoped seeding

Quand on crée un profile pour (`student`, `course`), on seed depuis le `_global` profile :

```

async def get_or_create(self, session_id, ..., course_id=None):
    if course_id and course_id != "_global":
        # Try course-specific first
        existing = await self._fetch(session_id, course_id)
        if existing:
            return existing
        # Seed from _global
        global_profile = await self._fetch(session_id, "_global")
        if global_profile:
            seeded = StudentProfile(**asdict(global_profile))
            seeded.course_id = course_id
            await self.save(seeded, course_id=course_id)
            log.info("Created course-scoped profile %s/%s seeded from global", ...)
            return seeded
    # Fall through to default creation
    ...

```

[Continued: sections 16-28 follow same pattern. See `SMART_TEACHER_ARCHITECTURE.md` for sections that didn't fit here in detail.]

16. pedagogy/personalization/tts_adapter.py

Rôle : composer le TTS rate (3 couches + override). Voir §13 et §10.6 du doc Architecture pour détails complets.

Structure clé du fichier :

```
# Layer 1 – defaults par niveau étudiant
_LEVEL_DEFAULT_RATE = {
    "collège": 0.92, "lycée": 1.00, "université": 1.05, "master": 1.05, ...
}

# Layer 2 – bandit modulation
_BANDIT_RATE_OFFSET = 0.15
_BANDIT_RATE_MULTIPLIERS = {
    "slow": 1.0 - _BANDIT_RATE_OFFSET,    # 0.85
    "normal": 1.0,
    "fast": 1.0 + _BANDIT_RATE_OFFSET,    # 1.15
}

# Layer 3 – confusion-driven slowdown
CONFUSION_TTS_SLOWDOWN_THRESHOLD = 0.6
CONFUSION_TTS_SLOWDOWN_FACTOR = 0.9
```

Fonction principale :

```
async def get_edge_tts_rate_with_bandit(session_id, bandit_speech_rate=None, confusion_score=None) -> str:
    try:
        params = await compute_tts_params(session_id)
        base_rate = float(params.get("rate", 1.0))
        manual_override = bool(params.get("manual_override", False))

        if manual_override:
            log.info("■■■ tts rate USER-OVERRIDE | rate=%.2f → %s (bandit/confusion ignored)", ...)
            return rate_float_to_edge_str(base_rate)

        bandit_mult = bandit_rate_multiplier(bandit_speech_rate)
        confusion_mult = confusion_rate_multiplier(confusion_score)
        final_rate = base_rate * bandit_mult * confusion_mult
        log.info("■■■ tts rate composed | profile=%.2f × bandit=%.2f × confusion=%.2f = %.2f → %s", ...)
        return rate_float_to_edge_str(final_rate)
    except Exception:
        return "+0%"
```

17. pedagogy/personalization/bandit/thomps on.py

Rôle : Thompson sampling sur Beta(α , β) per arm.

```
@dataclass
class ArmState:
    alpha: float = 1.0    # successes + 1
    beta: float = 1.0     # failures + 1

    @property
    def n_pulls(self) -> int:
        return int(round(self.alpha + self.beta - 2))

    @property
    def mean(self) -> float:
        return self.alpha / (self.alpha + self.beta)

    def sample(self, rng=None) -> float:
        rng = rng or random
        return rng.betavariate(self.alpha, self.beta)
```

```
def select_arm(arms_state: dict[str, ArmState], rng=None) -> str:
    rng = rng or random.Random()
    samples = {arm_name: arm.sample(rng) for arm_name, arm in arms_state.items()}
    chosen = max(samples, key=samples.get)
    # Log all samples
    sorted_samples = sorted(samples.items(), key=lambda kv: -kv[1])
    for arm_name, sample in sorted_samples[:5]:
        marker = " ← chosen" if arm_name == chosen else ""
        arm = arms_state[arm_name]
        log.info("■ arm=%-30s α=%.0f β=%.0f mean=%.2f sample=%.3f%s",
                arm_name, arm.alpha, arm.beta, arm.mean, sample, marker)
    return chosen

def update_arm(arms_state: dict[str, ArmState], chosen: str, reward: float) -> None:
    """α += reward, β += (1 - reward)."""
    if chosen not in arms_state:
        arms_state[chosen] = ArmState()
    arms_state[chosen].alpha += reward
    arms_state[chosen].beta += (1 - reward)
```

18. pedagogy/personalization/bandit/reward.py

Rôle : compute scalar reward from turn outcome.

```
W_CONFUSION = 0.50      # éviter de confuser = priorité
W_MASTERY = 0.30
W_ENGAGEMENT = 0.20

def compute_reward(outcome: TurnOutcome) -> float:
    confusion_signal = 0.0 if outcome.confused else 1.0
    mastery_signal = max(0.0, min(1.0, outcome.mastery_delta or 0.0))
    engagement_signal = 1.0 if outcome.engaged else 0.0

    reward = (
        W_CONFUSION * confusion_signal +
        W_MASTERY * mastery_signal +
        W_ENGAGEMENT * engagement_signal
    )
    log.info("bandit reward | confusion_signal=%.2f mastery_signal=%.2f engagement_signal=%.2f → reward=%.3f (",
            confusion_signal, mastery_signal, engagement_signal, reward,
            outcome.mastery_delta or 0.0, outcome.confused, outcome.engaged)
    return reward
```

19. pedagogy/resume_intelligence.py

Rôle : décider stratégie de reprise selon (cursor, narration_len, pause_duration).

Voir §11 du doc Architecture pour détails. Fonctions clés :

```
class ResumeIntent(str, Enum):
    QUICK_RESUME = "quick_resume"
    NORMAL_RESUME = "normal_resume"
    RESUME_AFTER_GAP = "resume_after_gap"
    RESUME_AFTER_LONG = "resume_after_long"
    SLIDE_COMPLETED = "slide_completed"
    REVIEW_REQUEST = "review_request"

class ResumeStrategy(str, Enum):
    CONTINUE = "continue"
```

```

REWIND_SENTENCE          = "rewind_sentence"
REWIND_SENTENCE_RECAP    = "rewind_sentence_recap"
REWIND_SENTENCE_LONG_RECAP = "rewind_sentence_long_recap"
REEXPLAIN_AND_CONTINUE   = "reexplain_and_continue"
SKIP_TO_NEXT             = "skip_to_next"

_INTENT_STRATEGY: dict[ResumeIntent, ResumeStrategy] = {
    ResumeIntent.QUICK_RESUME:      ResumeStrategy.CONTINUE,
    ResumeIntent.NORMAL_RESUME:     ResumeStrategy.REWIND_SENTENCE,
    ResumeIntent.RESUME_AFTER_GAP:   ResumeStrategy.REEXPLAIN_AND_CONTINUE,
    ResumeIntent.RESUME_AFTER_LONG:  ResumeStrategy.REEXPLAIN_AND_CONTINUE,
    ResumeIntent.SLIDE_COMPLETED:    ResumeStrategy.SKIP_TO_NEXT,
    ResumeIntent.REVIEW_REQUEST:     ResumeStrategy.REWIND_SENTENCE,
}

def detect_resume_intent(ctx: ResumeContext) -> ResumeIntent:
    log.info("■ detect_resume_intent | cursor=%d/%d (%.1f%%) pause=%s",
            ctx.cursor, ctx.narration_len,
            (ctx.cursor / ctx.narration_len * 100.0) if ctx.narration_len else 0.0,
            f"{ctx.interruption_duration_s:.1f}s" if ctx.interruption_duration_s else "?")

    if ctx.narration_len > 0 and ctx.cursor >= ctx.narration_len:
        if ctx.interruption_duration_s is not None and ctx.interruption_duration_s < SLIDE_DONE_GRACE_S:
            return ResumeIntent.SLIDE_COMPLETED
        return ResumeIntent.REVIEW_REQUEST

    bucket = _pause_bucket(ctx.interruption_duration_s)
    intent = {
        "quick": ResumeIntent.QUICK_RESUME,
        "normal": ResumeIntent.NORMAL_RESUME,
        "gap": ResumeIntent.RESUME_AFTER_GAP,
        "long": ResumeIntent.RESUME_AFTER_LONG,
    }.get(bucket, ResumeIntent.RESUME_AFTER_LONG)
    log.info("■ detect_resume_intent → %s (bucket=%s)", intent.value, bucket)
    return intent

```

20. services/presentation.py

Couvre `decide_narration_cache_reuse`, `compute_text_cursor_from_audio_progress`, `current_sentence_span`, `rewind_to_current_sentence_start`, `synthesize_cached_tts`. Voir §11 et §22 du doc Architecture.

Logique de cache decision :

- Path (1) : in-memory hit (`current_presentation_key == requested_slide_key`)
- Path (2) : Redis snapshot pour la slide demandée — avec **revisit-completed-slide guard**
- Path (3) : `paused_state` cold-cache fallback
- Sinon : MISS → régénération LLM

21-28. Voir doc Architecture

Ces fichiers sont déjà couverts en profondeur dans `SMART_TEACHER_ARCHITECTURE.md` (sections 6-9 pour les graphes, §8 pour RAG, §7 pour KG, §3 pour auth, etc.). Les patterns sont :

- `rag/multimodal_rag.py` : ~3 000 lignes. Les fonctions clés sont `_chunk_by_ideas`, `_summarise_chunks`, `_documents_from_course_data`, `_vector_search`, `_bm25_search`, `_rrf_fuse`, `_rerank_with_cross_encoder`, `_make_chat_llm`, `retrieve_chunks`, `run_ingestion_pipeline_from_course_data`. Toutes documentées dans le doc Architecture §8.
- `pedagogy/knowledge_graph/graph.py` : ~450 lignes. `IdeaNode`, `ConceptInfo`, `KnowledgeGraph` avec méthodes `add_node`, `add_documents`, `prerequisites_of`, `dependents_of`, `examples_of`, `learning_path`, `attach_concepts`, `list_concepts`, `concepts_of_idea`, `prereq_concepts`, `count_idea_edges_between`. Voir §9 du doc Architecture.
- `pedagogy/concept_from_titles.py` : `ConceptFromTitles` avec `extract`, `_group_by_title`, `_is_valid_title`, `_merge_similar_titles`, `_build_concepts`, `_llm_enrich`. Voir §7 et §9 du doc Architecture.
- `agentic/qa/intent.py` : SIGHT-first classifier + LLM JSON intent. `IntentAgent.__call__(state)` → modifie `state.intent`.
- `agentic/qa/responder.py` : ~1 800 lignes. `ResponderAgent` avec construction du prompt par mode (question / clarification / confusion_signal / navigation / off_topic). `_build_qa_prompt`, `_format_chunks_with_ids`, `_detect_leak_or_offtopic`, `_parse_qa_response`, `_extract_json`. Voir §6.5 du doc Architecture.
- `agentic/qa/reviewer.py` : `ReviewerAgent` → LLM Self-RAG → JSON `{grounded, feedback}`. Retry logic avec max 2 retries.
- `agentic/teaching/narrator.py` : `NarratorAgent` + post-processeurs (`_strip_greeting_opener`, `_strip_parenthetical_duplicates`, `_is_cliffhanger`).
- `handlers/ws.py` : ~3 800 lignes. Le plus gros fichier — gère tous les WebSocket events. Voir §13 du doc Architecture pour les message types et §3.7 pour la pause/resume flow.

Fin du document File-by-File Deep Dive.

Pour aller plus loin : référence-toi au doc principal `SMART_TEACHER_ARCHITECTURE.md` pour les diagrammes, schémas de données, formules détaillées, et logs.